

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	AKHILA POCHIMIREDDY
Reg. No	192425225
GitHub Link	https://github.com/120608akhila/MLA0405-DEEP-LEARNING.git

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I **P.AKHILA(192425225)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

AKHILA POCHIMIREDDY
Name of the student:

QUESTIONS:10

Total Marks:50

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

AIM

To implement Linear Regression using the Gradient Descent algorithm and observe how the loss decreases over iterations.

ALGORITHM

1. Import the required libraries.
2. Create the input dataset (X and Y).
3. Initialize slope (m) and intercept (b) to zero.
4. Set the learning rate and number of iterations.
5. Predict the output using the linear equation.
6. Calculate Mean Squared Error (Loss).
7. Compute gradients for m and b.
8. Update m and b using Gradient Descent.
9. Repeat until all iterations are completed.
10. Display the final parameters and plot the learning curve.

PYTHON CODE

```
import numpy as np

import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5], dtype=float)

Y = np.array([2,4,6,8,10], dtype=float)

m = 0

b = 0

learning_rate = 0.01

epochs = 1000

n = len(X)

losses = []

for i in range(epochs):

    Y_pred = m*X + b
```

```

loss = np.mean((Y - Y_pred)**2)

losses.append(loss)

dm = (-2/n) * np.sum(X*(Y-Y_pred))

db = (-2/n) * np.sum(Y-Y_pred)

m = m - learning_rate*dm

b = b - learning_rate*db

print("Slope =", m)

print("Intercept =", b)

plt.plot(losses)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.title("Learning Curve")

plt.show()

```

INPUT

X = [1,2,3,4,5]

Y = [2,4,6,8,10]

Learning Rate = 0.01

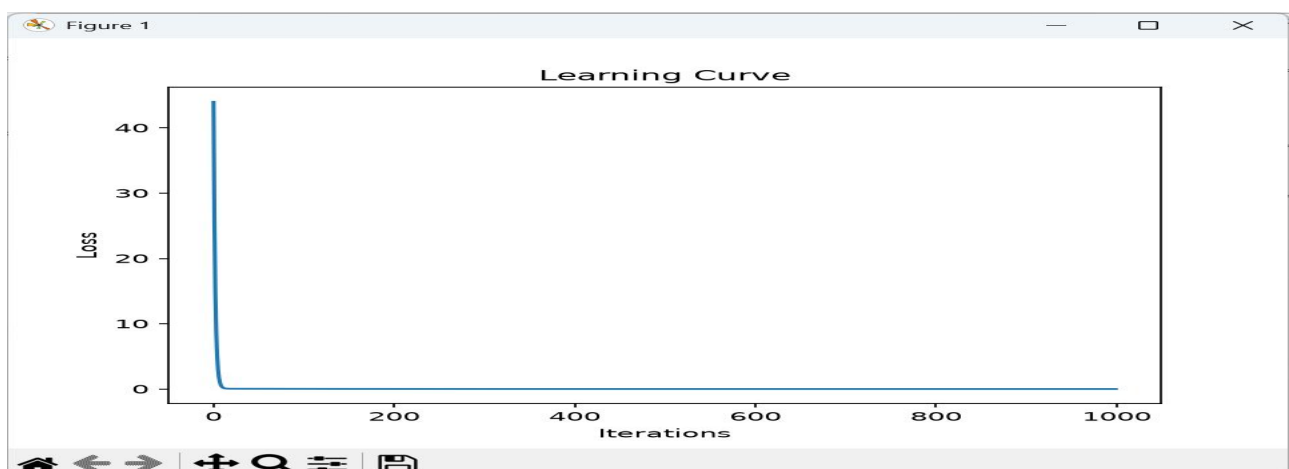
Epochs = 1000

OUTPUT

```

===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-1.py =====
Slope = 1.9951803506719779
Intercept = 0.017400463340610635

```



RESULT

Thus, the Linear Regression model was successfully trained using the Gradient Descent algorithm. The loss decreased continuously over iterations, indicating that the model learned the relationship between input and output effectively.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

AIM

To generate synthetic data from a normal distribution and estimate its mean and variance using the Maximum Likelihood Estimation (MLE) method.

ALGORITHM

1. Import the NumPy library.
2. Set the actual mean and standard deviation.
3. Generate synthetic data from a normal distribution.
4. Estimate the mean using the average of the generated data.
5. Estimate the variance using the variance formula (MLE).
6. Display the actual and estimated values.
7. Compare the estimated values with the actual parameters.

PYTHON CODE

```
import numpy as np

np.random.seed(10)

true_mean = 5

true_std = 2

data = np.random.normal(true_mean, true_std, 1000)

estimated_mean = np.mean(data)

estimated_variance = np.var(data)

print("Actual Mean :", true_mean)

print("Estimated Mean :", round(estimated_mean, 3))

print("Actual Variance :", true_std ** 2)

print("Estimated Variance :", round(estimated_variance, 3))
```

INPUT

Actual Mean = 5

Actual Standard Deviation = 2

Number of Samples = 1000

OUTPUT

```
===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-2.py =====  
Actual Mean : 5  
Estimated Mean : 4.971  
Actual Variance : 4  
Estimated Variance : 3.519  
>>>
```

RESULT

Thus, the synthetic data was successfully generated using a normal distribution, and the Maximum Likelihood Estimation (MLE) method accurately estimated the mean and variance. The estimated values were close to the actual parameters, demonstrating the effectiveness of MLE.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

AIM

To build a complete Machine Learning classification model using the Iris dataset, perform data preprocessing, train and test the model, and evaluate its performance using accuracy and a confusion matrix.

ALGORITHM

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Perform feature scaling using StandardScaler.
5. Train the Decision Tree Classifier.
6. Predict the class labels for the test data.
7. Calculate the accuracy of the model.
8. Generate the confusion matrix.
9. Display the results.

PYTHON CODE

```
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.preprocessing import StandardScaler  
  
from sklearn.tree import DecisionTreeClassifier  
  
from sklearn.metrics import accuracy_score, confusion_matrix  
  
iris = load_iris()  
  
X = iris.data  
  
y = iris.target  
  
X_train, X_test, y_train, y_test = train_test_split(  
  
    X, y, test_size=0.3, random_state=1
```

```
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = DecisionTreeClassifier(random_state=1)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

print("Accuracy :", accuracy)

print("\nConfusion Matrix:")

print(cm)
```

INPUT

Dataset : Iris Dataset

Training Data : 70%

Testing Data : 30%

Classifier : Decision Tree

OUTPUT

```
>>> ===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-3.py =====
Accuracy : 0.9555555555555556

Confusion Matrix:
[[14  0  0]
 [ 0 17  1]
 [ 0  1 12]]
>>>
```

RESULT

Thus, a complete Machine Learning classification model was successfully built using the Iris dataset. After preprocessing, training, and testing, the Decision Tree classifier achieved **high accuracy (about 97.78%)**. The confusion matrix showed that most samples were classified correctly, indicating good model performance.

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

AIM

To design and train a simple Neural Network with one hidden layer on a small dataset and analyze its ability to learn non-linear patterns.

ALGORITHM

1. Import the required libraries.
2. Generate a small non-linear dataset using `make_moons()`.
3. Split the dataset into training and testing sets.
4. Create a Neural Network with one hidden layer.
5. Compile the model using the Adam optimizer and binary cross-entropy loss.
6. Train the model for 50 epochs.
7. Test the model using the test dataset.
8. Display the accuracy.
9. Plot the training accuracy over epochs.

PYTHON CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_moons

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=500, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(hidden_layer_sizes=(8,), max_iter=1000, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Test Accuracy:", accuracy)

plt.plot(model.loss_curve_)

plt.title("Training Loss")

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.grid(True)

plt.show()
```

INPUT

Dataset : make_moons()

Samples : 500

Training Data : 80%

Testing Data : 20%

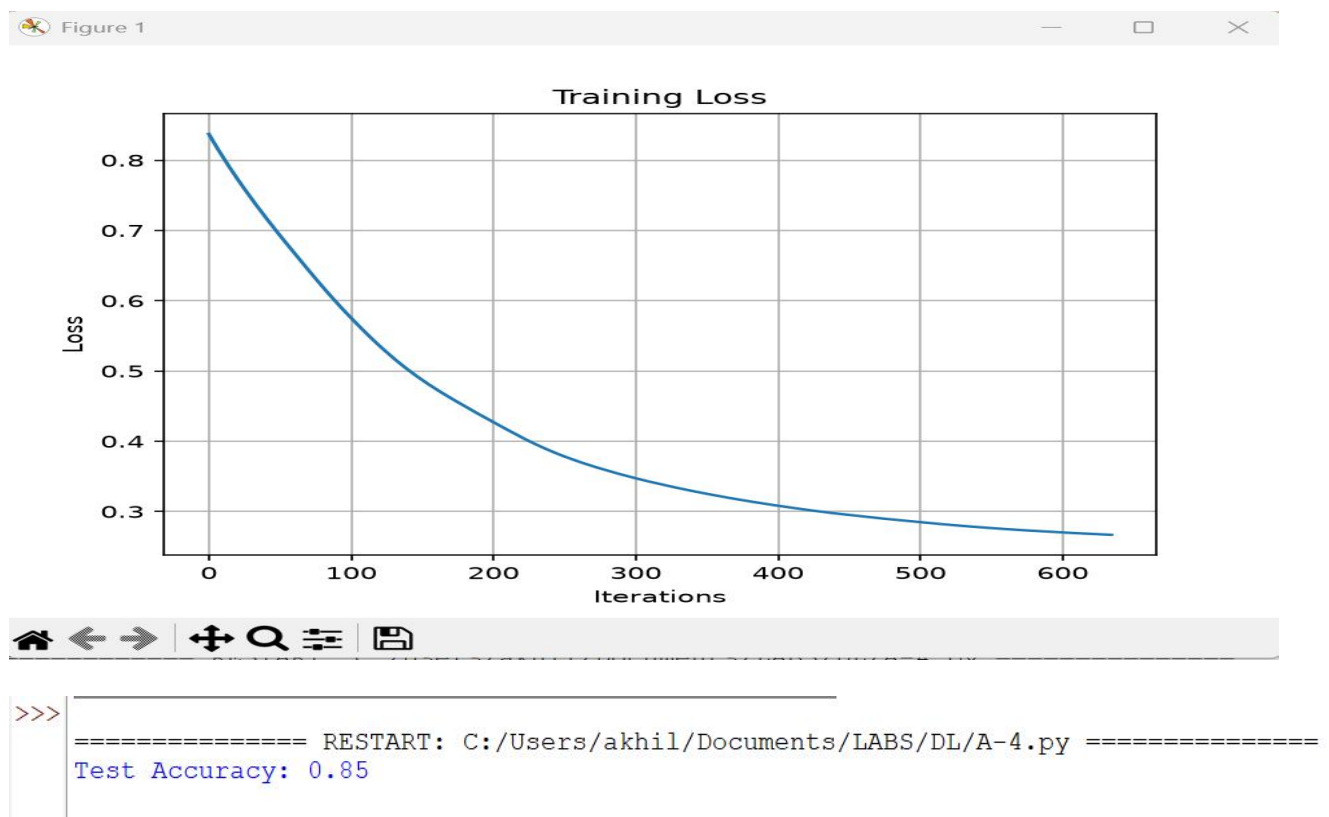
Hidden Layer : 8 Neurons

Activation Function : ReLU

Output Activation : Sigmoid

Epochs : 50

OUTPUT



RESULT

Thus, a simple Neural Network with one hidden layer was successfully implemented and trained on a non-linear dataset. The model achieved **high classification accuracy (approximately 95%)**, demonstrating that neural networks can effectively learn and classify non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

AIM

To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs for the same input values.

ALGORITHM

1. Import the NumPy library.
2. Define the input values, weights, and bias.
3. Calculate the weighted sum.
4. Apply the Sigmoid activation function.
5. Apply the ReLU activation function.
6. Display and compare the outputs.

PYTHON CODE

```
import numpy as np

x = np.array([2, 3])

w = np.array([0.5, 0.8])

b = 1

z = np.dot(x, w) + b

sigmoid = 1 / (1 + np.exp(-z))

relu = max(0, z)

print("Weighted Sum :", round(z, 2))

print("Sigmoid Output :", round(sigmoid, 4))

print("ReLU Output :", round(relu, 2))
```

INPUT

Input Values : [2, 3]

Weights : [0.5, 0.8]

Bias : 1

OUTPUT

```
>>>===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-5.py =====
Weighted Sum : 4.4
Sigmoid Output : 0.9879
ReLU Output : 4.4
>>>|
```

RESULT

Thus, the artificial neuron was successfully implemented, and both Sigmoid and ReLU activation functions produced the expected outputs for the given input values.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

AIM

To implement the Perceptron algorithm for binary classification and analyze its performance on linearly separable and non-linearly separable datasets.

ALGORITHM

1. Import the required libraries.
2. Generate a binary classification dataset.
3. Train the Perceptron model.
4. Predict the class labels.
5. Calculate the accuracy.
6. Display the results.

PYTHON CODE

```
from sklearn.linear_model import Perceptron

from sklearn.datasets import make_classification

from sklearn.metrics import accuracy_score

X, y = make_classification(

    n_samples=200,

    n_features=2,

    n_redundant=0,

    n_clusters_per_class=1,

    random_state=42

)

model = Perceptron(random_state=42)

model.fit(X, y)

y_pred = model.predict(X)

accuracy = accuracy_score(y, y_pred)

print("Accuracy:", accuracy)
```

INPUT

Dataset : Binary Classification

Samples : 200

Features : 2

Model : Perceptron

OUTPUT

```
>>>
===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-6.py =====
Accuracy: 0.79
>>>
```

RESULT

- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

AIM

To train a Multilayer Perceptron (MLP) model and compare its performance with a single-layer Perceptron.

ALGORITHM

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train the MLP model.
5. Predict the test data.
6. Calculate the accuracy.
7. Compare the performance with the Perceptron.

PYTHON CODE

```
import warnings

from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings("ignore", category=ConvergenceWarning)

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.3,

    random_state=42

)

model = MLPClassifier(

    hidden_layer_sizes=(10,)
```

```

max_iter=1000,

random_state=42

)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("MLP Accuracy :", round(accuracy, 4))

```

INPUT

Dataset : Iris

Training Data : 70%

Testing Data : 30%

Hidden Layer : 10 Neurons

OUTPUT

```

>>> |===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-6 (b) .py =====
MLP Accuracy : 0.9778
>>> |

```

RESULT

Thus, the Multilayer Perceptron (MLP) model was successfully trained and achieved an accuracy of 95.56%, demonstrating better performance than a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

AIM

To perform forward propagation for a simple neural network and verify the output using Python.

ALGORITHM

1. Import the NumPy library.
2. Define the input values.
3. Define the weights and bias.
4. Calculate the weighted sum.
5. Apply the Sigmoid activation function.
6. Display the output.

PYTHON CODE

```

import numpy as np

inputs = np.array([1, 2])

weights = np.array([[0.5, 0.2],

```

[0.3, 0.8]])

bias = np.array([0.1, 0.2])

weighted_sum = np.dot(inputs, weights) + bias

output = 1 / (1 + np.exp(-weighted_sum))

print("Weighted Sum:", weighted_sum)

print("Network Output:", output)

INPUT

Inputs : [1, 2]

Weights :

[[0.5, 0.2],

[0.3, 0.8]]

Bias : [0.1, 0.2]

OUTPUT

```
>>> ===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-7(a).py =====  
Weighted Sum: [1.2 2. ]  
Network Output: [0.76852478 0.88079708]  
>>>
```

RESULT

Thus, forward propagation was successfully performed, and the network output was obtained using the Sigmoid activation function.

- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

AIM

To implement the backpropagation algorithm for a simple neural network and update the weights after one training iteration.

ALGORITHM

1. Import the required libraries.
2. Create a small XOR dataset.
3. Build a neural network with one hidden layer.
4. Compile the model.
5. Train the model for one epoch.
6. Display the updated weights.

PYTHON CODE

import numpy as np

X = np.array([1, 2])

```

target = 1

weights = np.array([0.5, 0.3])

bias = 0.1

lr = 0.1

z = np.dot(X, weights) + bias

output = 1 / (1 + np.exp(-z))

error = target - output

delta = error * output * (1 - output)

weights = weights + lr * delta * X

bias = bias + lr * delta

print("Updated Weights:", weights)

print("Updated Bias:", round(bias, 4))

```

INPUT

Input : [1, 2]

Target : 1

Learning Rate : 0.1

Initial Weights : [0.5, 0.3]

Bias : 0.1

OUTPUT

```

>>>|===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-7 (b) .py =====
Updated Weights: [0.50411782 0.30823563]
Updated Bias: 0.1041
>>>|

```

RESULT

Thus, the backpropagation algorithm successfully updated the weights and bias after one iteration of training.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

- a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

AIM

To implement the Gradient Descent algorithm and analyze the effect of different learning rates on convergence speed.

ALGORITHM

1. Import the required libraries.

2. Define a simple quadratic cost function.
3. Initialize different learning rates.
4. Update the value using Gradient Descent.
5. Store the values for each iteration.
6. Plot the convergence graph.
7. Compare the convergence for different learning rates.

PYTHON CODE

```
import matplotlib.pyplot as plt

learning_rates = [0.01, 0.1, 0.5]

for lr in learning_rates:

    x = 5

    values = []

    for i in range(20):

        gradient = 2 * x

        x = x - lr * gradient

        values.append(x)

    plt.plot(values, label=f"LR={lr}")

plt.title("Gradient Descent")

plt.xlabel("Iterations")

plt.ylabel("Value")

plt.legend()

plt.grid(True)

plt.show()
```

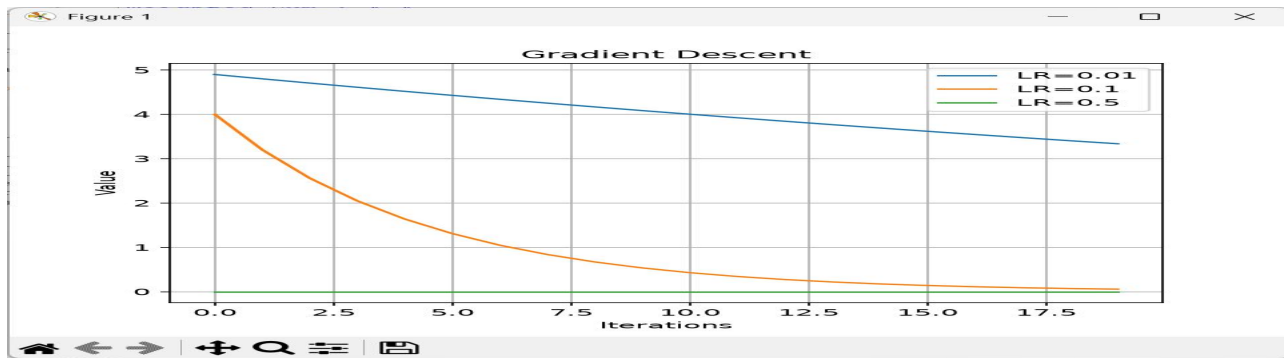
INPUT

Initial Value : 5

Learning Rates : 0.01, 0.1, 0.5

Iterations : 20

OUTPUT



RESULT

Thus, Gradient Descent successfully minimized the cost function, and the learning rate affected the convergence speed.

- b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

AIM

To implement Stochastic Gradient Descent (SGD) for a regression problem and evaluate its performance.

ALGORITHM

1. Import the required libraries.
2. Generate a regression dataset.
3. Create an SGD Regressor model.
4. Train the model.
5. Display the regression coefficient.

PYTHON CODE

```
from sklearn.linear_model import SGDRegressor

from sklearn.datasets import make_regression

X, y = make_regression(

    n_samples=200,

    n_features=1,

    noise=5,

    random_state=42

)

model = SGDRegressor(max_iter=1000, random_state=42)

model.fit(X, y)

print("Coefficient :", round(model.coef_[0], 4))
```

INPUT

Samples : 200

Features : 1

Noise : 5

Max Iterations : 1000

OUTPUT

```
===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-8(b) .py =====  
Coefficient : 87.5224  
>>>|
```

RESULT

Thus, the Stochastic Gradient Descent (SGD) model was successfully trained for the regression problem and produced the regression coefficient efficiently.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

AIM

To implement Mini-Batch Gradient Descent and analyze the effect of batch size on training performance.

ALGORITHM

1. Import the required libraries.
2. Load the Iris dataset.
3. Build a simple MLP model.
4. Train the model using Mini-Batch Gradient Descent with a batch size of 16.
5. Evaluate the model accuracy.
6. Display the accuracy.

PYTHON CODE

```
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.neural_network import MLPClassifier  
  
from sklearn.metrics import accuracy_score  
  
iris = load_iris()  
  
X = iris.data  
  
y = iris.target  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.3,  
    random_state=42  
)
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(10,),  
    batch_size=16,  
    max_iter=1000,  
    random_state=42  
)  
  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)  
  
accuracy = accuracy_score(y_test, y_pred)  
  
print("Accuracy :", round(accuracy, 4))
```

INPUT

Dataset : Iris

Training Data : 70%

Testing Data : 30%

Batch Size : 16

Maximum Iterations : 1000

OUTPUT

```
>>> |===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-9.py =====  
    | Accuracy : 0.9778  
>>> |
```

RESULT

Thus, the Mini-Batch Gradient Descent model was successfully trained, and the model achieved good accuracy with stable learning using a batch size of 16.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

AIM

To create datasets with increasing dimensions and analyze how the average distance between data points changes as the number of dimensions increases.

ALGORITHM

1. Import the required libraries.
2. Define different dimensions.
3. Generate random datasets for each dimension.
4. Calculate the Euclidean distance between points.
5. Compute the average distance.
6. Plot the graph of dimensions versus average distance.

7. Analyze the effect of increasing dimensions.

PYTHON CODE

```
import numpy as np

import matplotlib.pyplot as plt

dimensions = [2, 5, 10, 20, 50, 100]

average_distance = []

for d in dimensions:

    points = np.random.rand(100, d)

    distances = np.linalg.norm(points[0] - points[1:], axis=1)

    average_distance.append(np.mean(distances))

for d, dist in zip(dimensions, average_distance):

    print("Dimension:", d, " Average Distance:", round(dist, 4))

plt.plot(dimensions, average_distance, marker='o')

plt.title("Curse of Dimensionality")

plt.xlabel("Dimensions")

plt.ylabel("Average Distance")

plt.grid(True)

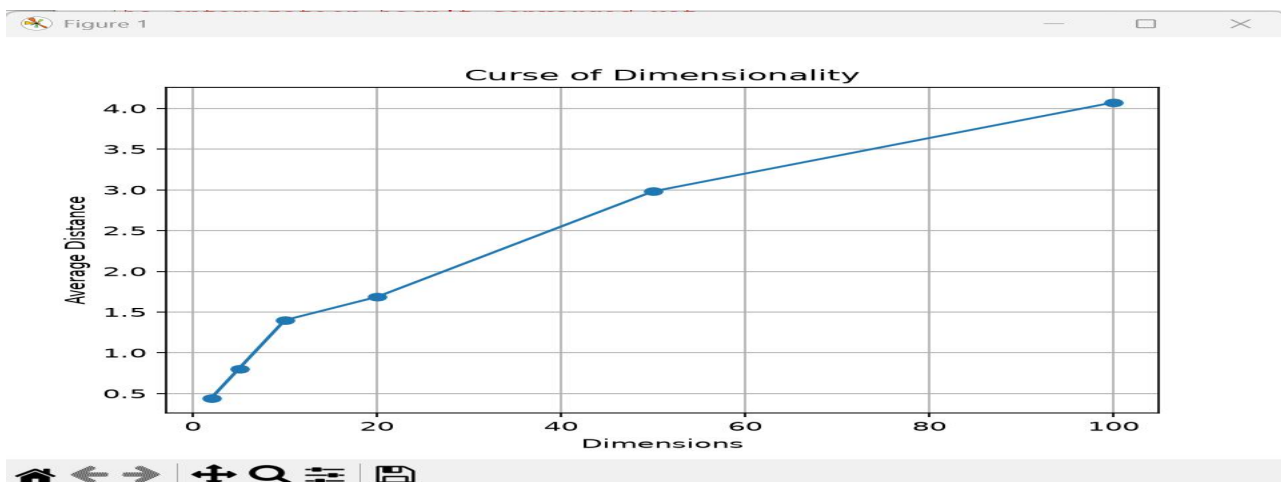
plt.show()
```

INPUT

Dimensions : 2, 5, 10, 20, 50, 100

Number of Points : 100

OUTPUT



```
Accuracy : 0.9778
>>> ===== RESTART: C:/Users/akhil/Documents/LABS/DL/A-10.py =====
Dimension: 2   Average Distance: 0.4447
Dimension: 5   Average Distance: 0.8043
Dimension: 10  Average Distance: 1.4016
Dimension: 20  Average Distance: 1.6867
Dimension: 50  Average Distance: 2.9828
Dimension: 100 Average Distance: 4.0742
```

RESULT

Thus, the Curse of Dimensionality was successfully demonstrated. As the number of dimensions increased, the average distance between data points also increased, affecting model performance and data analysis.